
Press Esc to exit full screen
Digital Design Concepts
Mohamed Fawzy
Senior Software QA Engineer - Questa Static&Formal
Feb 2023
00:11
El-Kholy, Mohamed (DI SW ICS MNA RD QA QST 3)
KN

HDL: Hardware Description Language

- **Verilog - VHDL**

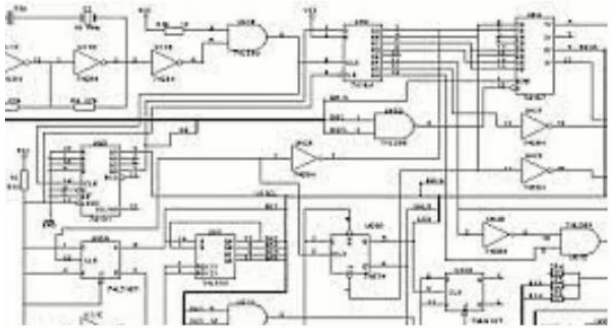
H Hardware

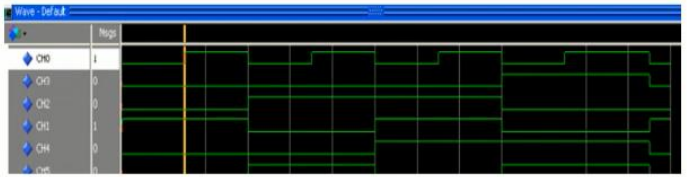
D Description

L Language

~~Software~~

~~Programming~~

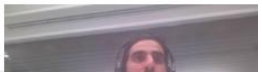




Session (Digital Design Concepts) - Outline

- HDL Vs Programming Language
- Modeling of Different Digital Circuits
  - Combinational circuits
  - Sequential circuits
  - Memory
  - FSM
- Good/Bad Practices in Digital Design
- Digital Design Metrics
- Design Optimization Techniques
  - Pipelining
  - Parallelism
  - Resource Sharing



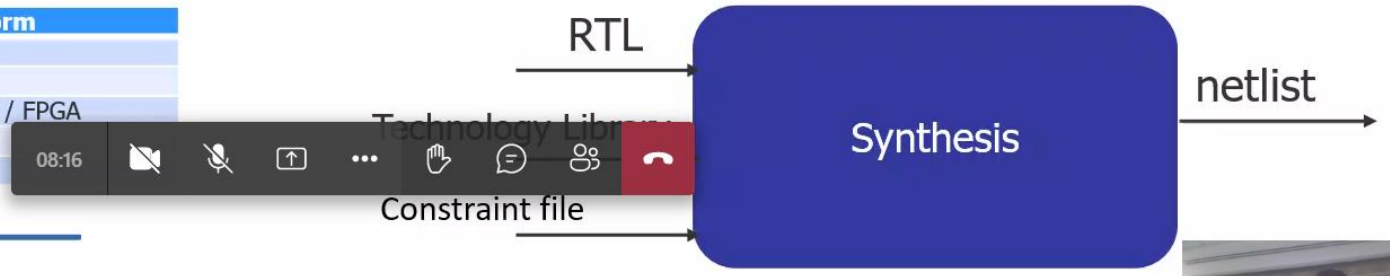


HDL Vs Programming Language

Hardware Description Language	Programming Language
concurrent in nature and executes a piece of code in **parallel**	**sequential** in nature and executes a piece of code **sequentially**.
often used to model a **Digital circuits** and the circuits **synthesized** to a hardware	often executed as a **piece of instructions** to CPU and the code is not synthesizable
knowledge of **Digital and hardware circuit** is needed	**Algorithm and Processor** knowledge is needed

- In digital circuit design, **register-transfer level (RTL)** is a design abstraction which models a synchronous digital circuit in terms of the flow of digital signals (data) between hardware registers, and the logical operations performed on those signals.
- Hardware implementation
  - Should match functionality with Simulation
  - Should be Optimized
- Synthesis converts Verilog code into optimized logic gates
  - Commercial Synthesis Tools:

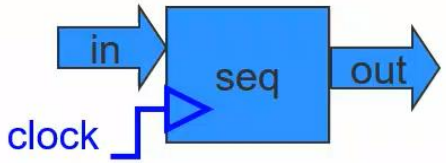
Vendor	Tool	Platform
Altera/INTEL	Quartus	FPGA
Xilinx	Vivado	FPGA
Siemens EDA	Precision	ASINC / FPGA
Synopsys	Design Compiler	ASIC
Synplicity	Synplify	ASIC



El-Kholy, Mohamed (DI SW ICS MNA RD QA QST 3)

Types of Digital Circuits





Combinational Circuit	*Sequential Circuit*
'out' is affected by the value of 'in' in no time	*Propagation* of 'in' to 'out' depends on *'clock'*
'out' is a function of the current value 'in'	*'out' is a function of the history of 'in'*
No storage needed	*Storage* is needed Two Types : FF , Latch
Example : Adder - Mux	*Example : Counter*

Modeling of Combinational Circuit

- Using “assign”

```
module half_adder(S, C, A, B);
   output S, C;
   input    A, B;

   wire S, C, A, B;

   assign S = A ^ B;
   assign C = A & B;

endmodule
```

- Using always

```
always @(in or sel)
   if (sel == 0)
      out = in[0];
   else if (sel == 1)
      out = in[1];
   else if (sel == 2)
      out = in[2];
   else
      out = in[3];
```

- Using gates primitives

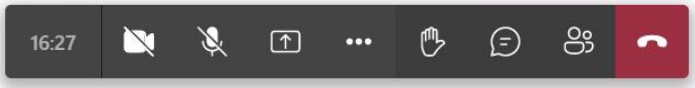
```
module half_adder(S, C, A, B);
   output S, C;
   input A, B;

   wire S, C, A, B;

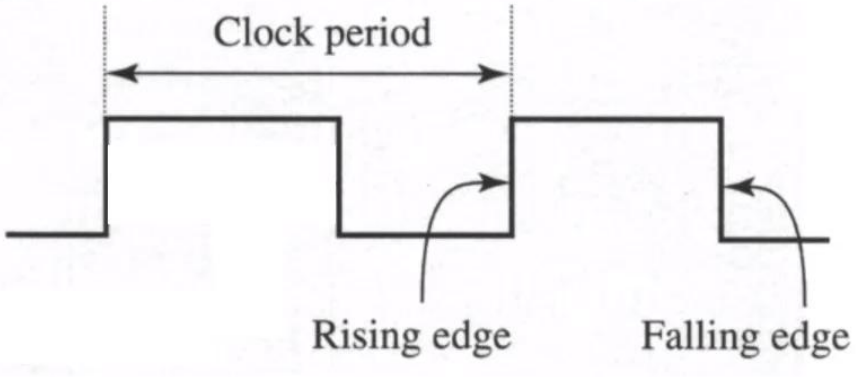
   xor  #2 (S, A, B);
   and #1 (C, A, B);

endmodule
```

- When do we use the always block to describe a combinational logic?
  - Normally for high-complexity Comb. Logic
    - When output depends on several conditions, which requires if-else or case constructs to be fully described



What is Clock ?



- Controls signals propagation in the circuit
- Allows sequential elements to load values on:
  - Rising edge or Falling edge **(Flip Flops)**
  - level high or level low **(Latches)**

Modeling of Basic Sequential Elements

*Sensitivity list specifying the conditions which trigger the block

```
always @ ( posedge Clock )
        Qout1 <= Din ;
```

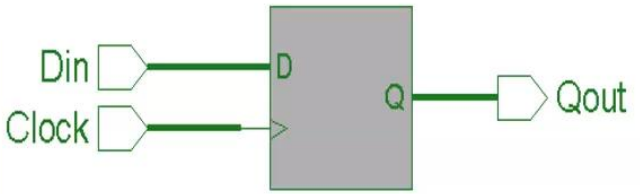
```
process(Clk)
begin
   if(rising_edge(Clk)) then
  Q <= D;
   end if;
end process;
```

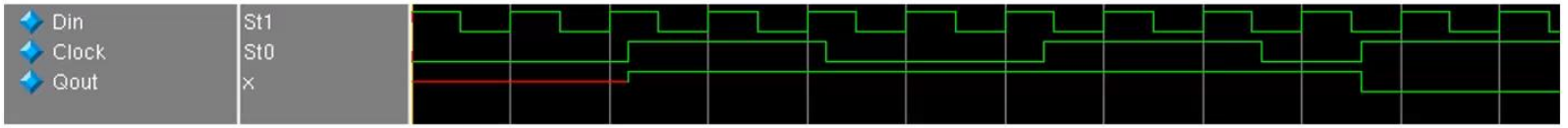
```
process(data_in, clock)
begin

    -- clock rising edge

    if (clock='1' and clock'event) then
        data_out <= data_in;
    end if;

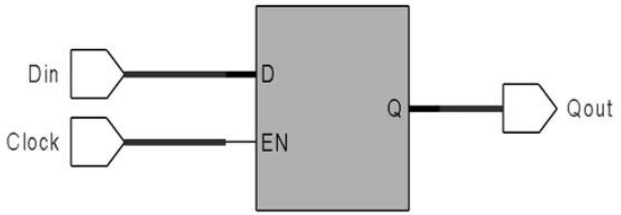
end process;
```

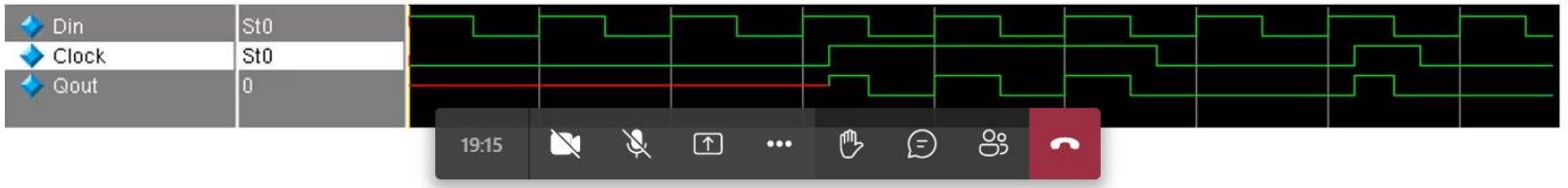





```
always @ ( Clock , Din )
  if ( Clock == 1'b1 )
    Qout <= Din ;
```

```
process(data_in, enable)
begin
    if (enable='1') then
        -- no clock signal here
        data_out <= data_in;
    end if;
end process;
```





El-Kholy, Mohamed (DI SW ICS MNA RD QA QST 3)

Resets

- Necessary for sequential circuits
- Put the circuit in a defined (start) state
  - i.e. load a defined value in all sequential elements (flipflops & latches)

	Asynchronous	Synchronous
Resetting time	Can be asserted at any point during the clock cycle.	Resets the state of the flip-flop on the active edge of a clock.
Sensitivity to Glitches 	Sensitive (disadvantage)	Insensitive (advantage)
Metastability	May have metastability issues due to multiple domains (disadvantage)	Gives a completely synchronous circuit (advantage)
Reset Length	Reset gets the highest priority, Does not require presence of clock to reset the circuit. (advantage)	Reset needs to be stretched, if it is not long enough to be seen at the active clock edge. (disadvantage)
Synthesis	Not an issue (advantage)	Difficult to differentiate reset from other signals, care must be taken during synthesis (disadvantage)
Practicality	• Asynchronous Resets more recommended • SoC designs are architected for reduced power consumption. • Achieved by turning off clock and power domains when they are unused. • Fail to capture the reset signal itself, for sufficiently slow clocks • Fully synchronous resets are not recommended Unless the capture reset assertion can be guaranteed by design. • A system with multiple clock domains requires the use of asynchronously reset flip-flops to: • Have the ability to reset flip-flops to a known value AND Be able to do so without the clock to those domains being active	

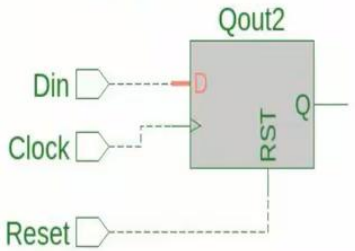
Resets

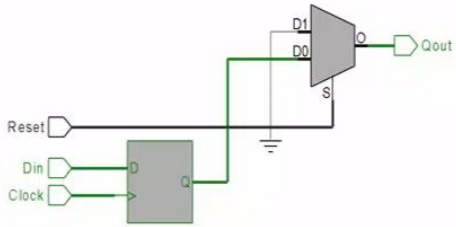
Modeling

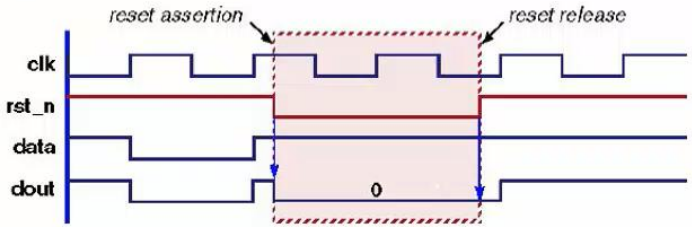
Asynchronous

```
always @ (posedge Clock or posedge Reset)
  if (Reset)  Qout2 <= 0 ;
  else        Qout2 <= Din ;
```

```
process(Clk,async_reset)
begin
    if(async_reset='1') then
  Q <= '0';
    elsif(rising_edge(Clk)) then
  Q <= D;
 end if;
end process;
```



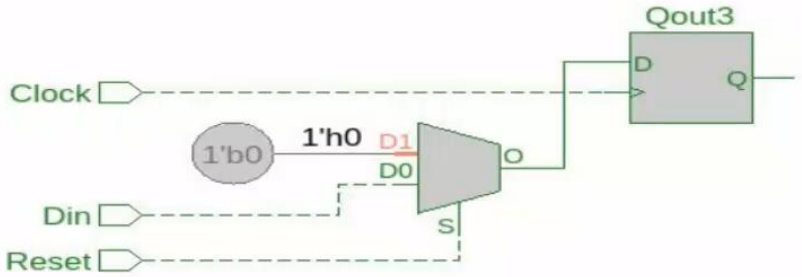


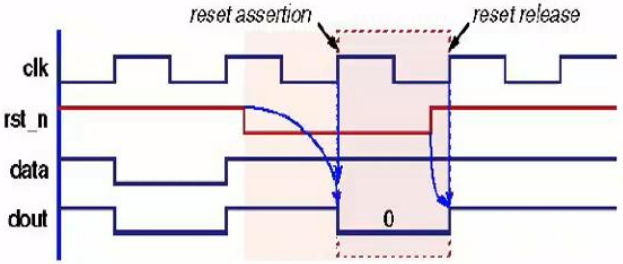


Synchronous

```
process(Clk)
begin
   if(rising_edge(Clk)) then
  if(sync_reset='1') then
   Q <= '0';
  else
   Q <= D;
  end if;
   end if;
end process;
```

```
always @ (posedge Clock)
  if (Reset)  Qout3 <= 0 ;
  else        Qout3 <= Din ;
```



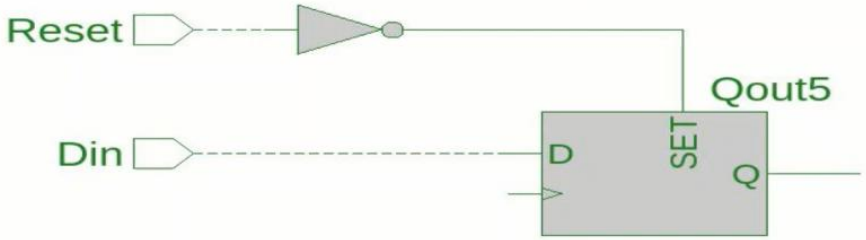


Modeling of Variations of Sequential Elements

- Reset Information
  - Polarity : Active High , Active Low , other_conditions (AL - Asynchronous Load)
  - Value : 0 (Reset) , 1 (Set) , other_values (AD - Asynchronous Data)

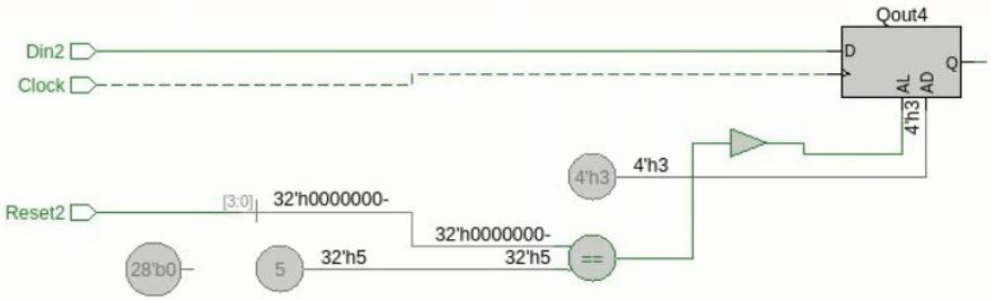
- FF with Active low Set

```
always @ (posedge Clock or negedge Reset)
  if (!Reset)  Qout5 <= 1 ;
  else         Qout5 <= Din ;
```



- FF with AL , AD

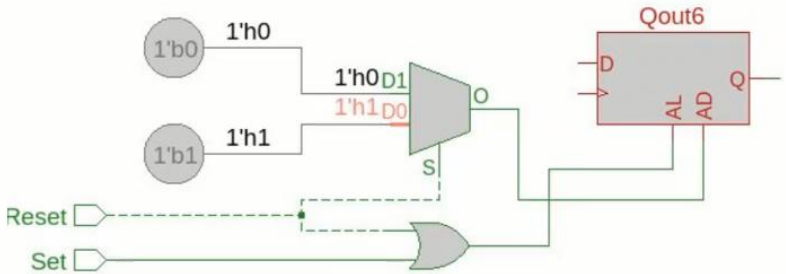
```
wire rst_cond ;
assign rst_cond = Reset2 ==5 ;
always @ (posedge Clock or posedge rst_cond)
  if (rst_cond)  Qout4 <= 3 ;
  else           Qout4 <= Din2 ;
```



Modeling of Variations of Sequential Elements

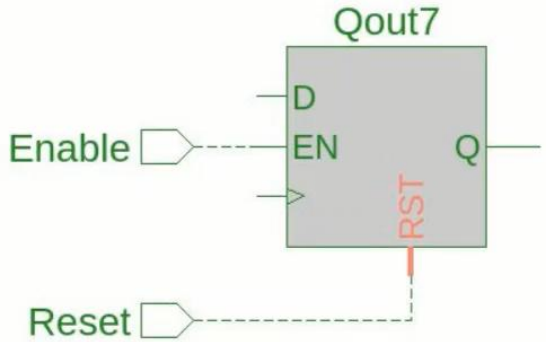
- FF with Priority Reset-Set

```
always @ (posedge Clock or posedge Reset or posedge Set)
  if (Reset)  Qout6 <= 0 ;
  else if (Set) Qout6 <= 1 ;
  else        Qout6 <= Din ;
```



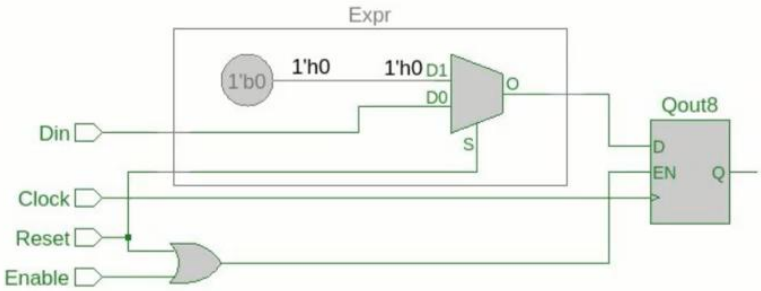
- FF with Asynchronous Reset and Synchronous Enable

```
always @ (posedge Clock or posedge Reset)
  if (Reset)  Qout7 <= 0 ;
  else if (Enable)       Qout7 <= Din ;
```



- FF with Synchronous Reset and Synchronous Enable

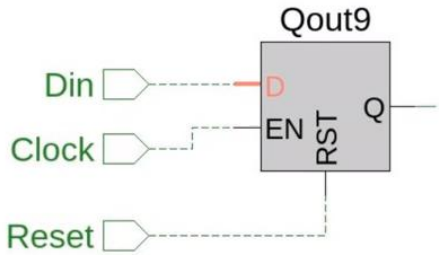
```
always @ (posedge Clock)
  if (Reset)  Qout8 <= 0 ;
  else if (Enable)       Qout8 <= Din ;
```



Modeling of Variations of Sequential Elements

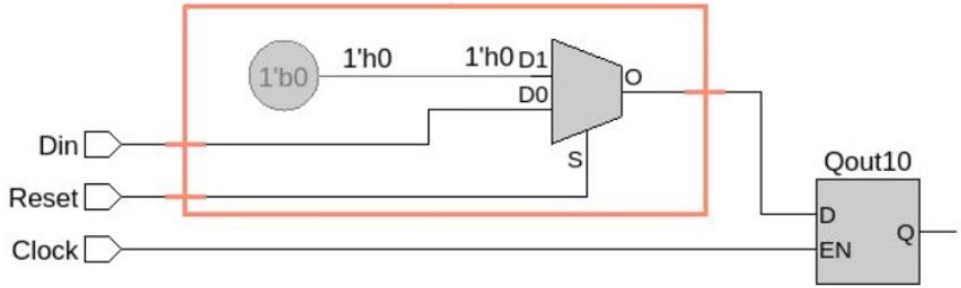
- Latch with Asynchronous Reset

```
always @ (Clock or Din or Reset)
  if (Reset)  Qout9 <= 0 ;
  else if (Clock)      Qout9 <= Din ;
```



- Latch with Synchronous Reset

```
always @ (Clock or Din)
  if (Clock) if (Reset) Qout10 <= 0 ; else  Qout10 <= Din ;
```

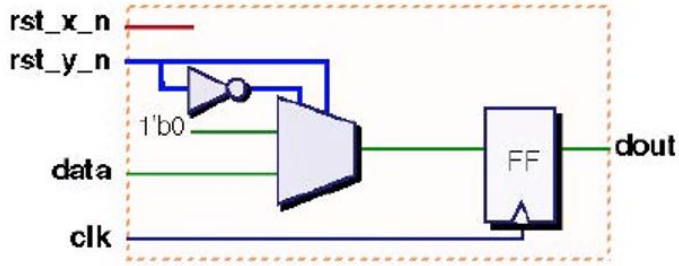


01:07:14

Modeling of Variations of Sequential Elements

- **Reset and No-op Reset**

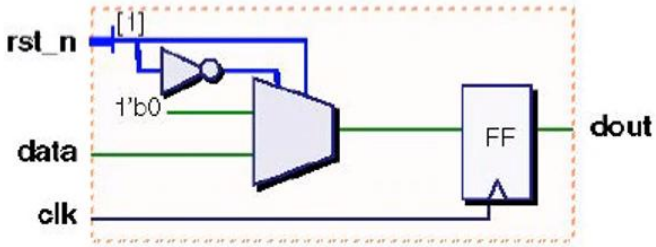
```
always @(posedge clk)begin
  if (!rst_x_n) dout <= 1'b0;
  else dout <= data ;
  if (!rst_y_n) dout <= 1'b0;
  else dout <= data ;
end
```



The first reset (*rst_x_n*) has no effect; it is not detected as a reset.

- **Bused Reset**

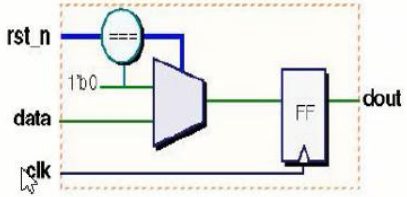
```
always @(posedge clk)
   if (!rst_n[1]) dout <= 1'b0
   else dout <= data;
```



01:08:43

Modeling of Variations of Sequential Elements

- **Equivalence Operator Reset**




```
always @(posedge clk)
   if (rst_n === 1'b0) dout <= 1'b0
   else dout <= data;
```

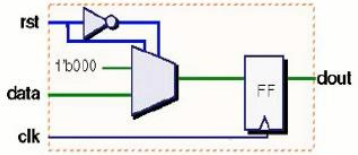
Reset activated with an equivalence operator (===, !===, == and !=) is detected as a reset.

- **Gated Clock**

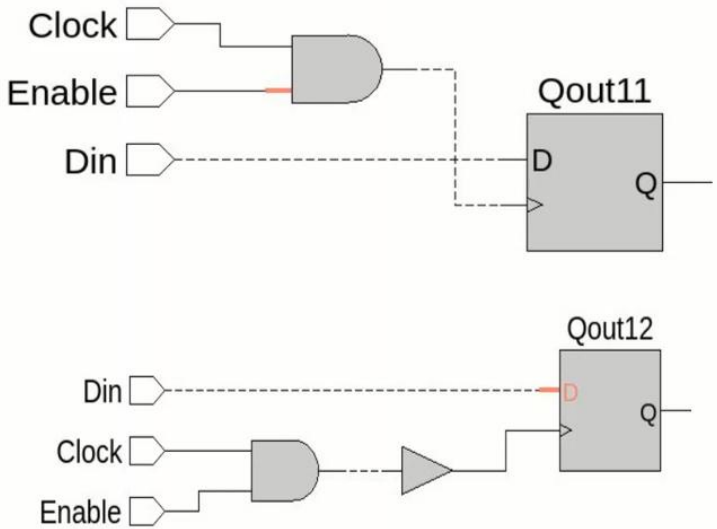
```
always @ (posedge (Clock & Enable ) )
   Qout11 <= Din ;

wire gclk ;
assign gclk = Clock & Enable ;
always @ (posedge gclk) Qout12 <= Din ;
```

- **Case Coded Reset for Multiple-bit Register**

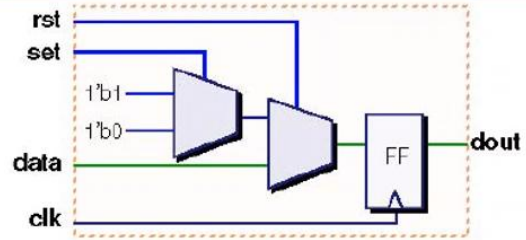



```
always @ ( posedge clk ) begin
   case (rst)
      0 : dout = data;     // multibit
      1 : dout = 1'b000;
   endcase
end
```

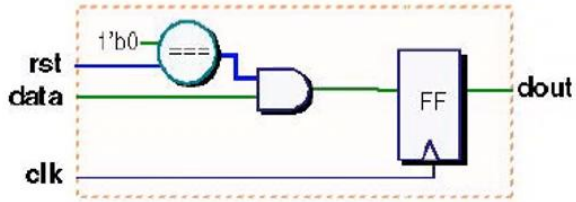


Modeling of Variations of Sequential Elements

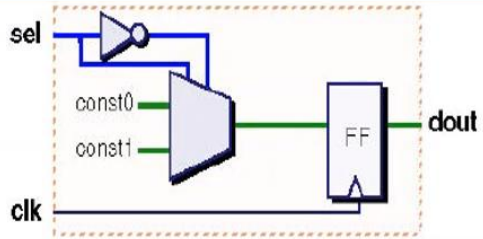
- More Styles




```
always @ (posedge clk)
   if (rst) begin
      dout <= 1'b0;
      if (set) dout <= 1'b1;
   end
   else dout <= data;
```




```
always @ (posedge clk) begin
   case (rst)
      0 : dout = data;     // 1-bit
      1 : dout = 1'b0;
   endcase
end
```




```
always @(posedge clk)
   if (sel) q <= const0;
   else q <= const1;
```

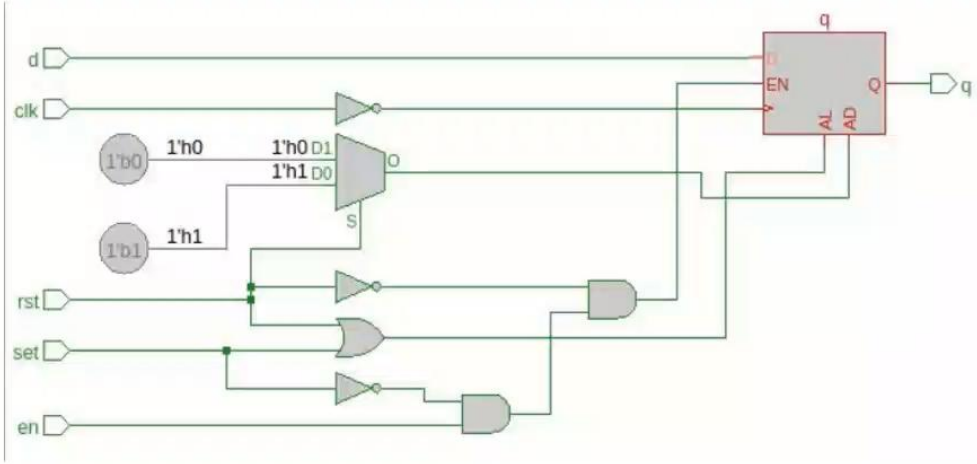
01:11:12

El Kholy, Mohamed (DI SW ICS MNA RD QA QST 3)

Modeling of Variations of Sequential Elements

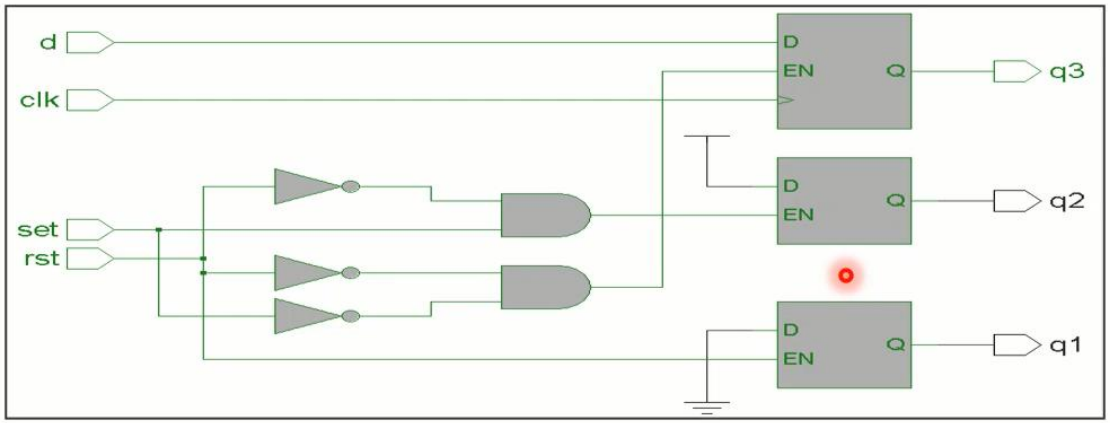
- More styles

```
process ( clk , set , rst, en )
begin
        if      ( rst = '1' ) then
                q <= '0';
        elsif   ( set = '1' ) then
                q <= '1';
        elsif   ( (en = '1') and ( clk = '0' and clk'event) ) then
                q <= d;
        end if;
end process;
```



Modeling of Variations of Sequential Elements

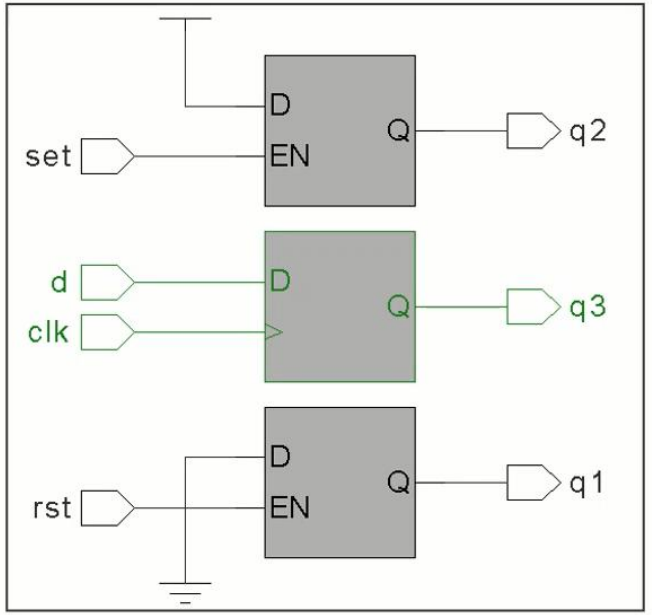
```
process ( clk , rst , set )
begin
        if      ( rst = '1' ) then
                q1 <= '0';
        elsif   ( set = '1' ) then
                q2 <= '1';
        elsif ( clk = '1' and clk'event ) then
                q3 <= d;
        end if;
end process;
```




```
process ( clk , rst , set )
begin
        if   ( rst = '1' ) then
                q1 <= '0';
        end if;

        if      ( set = '1' ) then
                q2 <= '1';
        end if;

        if ( clk'event and clk='1' ) then
                q3 <= d;
        end if;
end process;
```



Lab 1

```
process ( clk )
begin
        if ( clk = '1' and clk'event ) then
                temp := din ;
            dout <= temp ;
        end if;
end process ;
```

```
process ( clk )
begin
        if ( clk = '1' and clk'event ) then
                dout <= temp ;
            temp := din ;
        end if;
end process ;
```

```
process ( clk , din , temp )
begin
        if ( clk = '0' ) then
                temp <= din ;
                dout <= temp ;
        end if ;
end process ;
```

```
process ( rst , set , din )
begin
        if ( rst = '1' ) then
                var <= '0' ;
        elsif ( set = '1' ) then
                var <= '1' ;
        else
                var <= din ;
        end if ;
end process ;
```

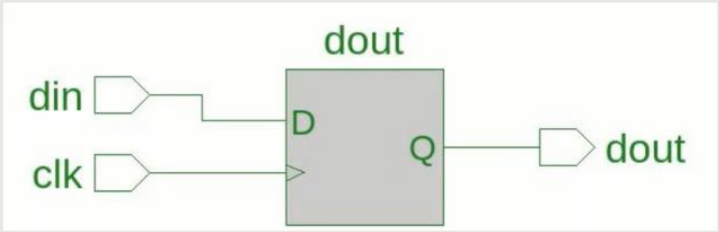
```
process ( rst , clk )
begin
        if ( rst = '0' ) then
                var <= '0' ;
        elsif ( Rising_edge(clk) ) then
                if ( set = '1' )  then
                        var <= '1' ;
                else
                        var <= din ;
                end if ;
        end if ;
end process ;
```

01:22:17

Lab 1 - Solutions

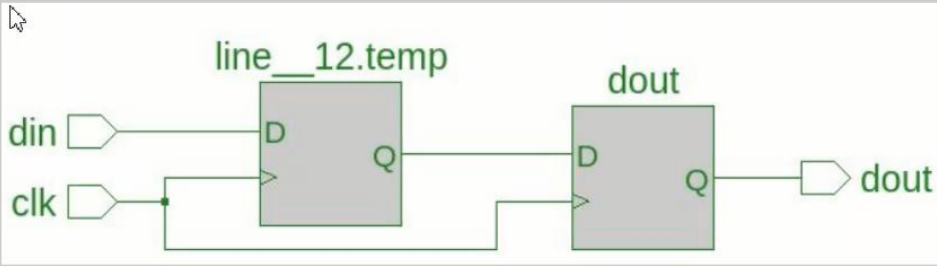
```
process ( clk )
begin
        if ( clk = '1' and clk'event ) then
                temp := din ;
            dout <= temp ;
        end if;
end process ;
```

Write Before Read

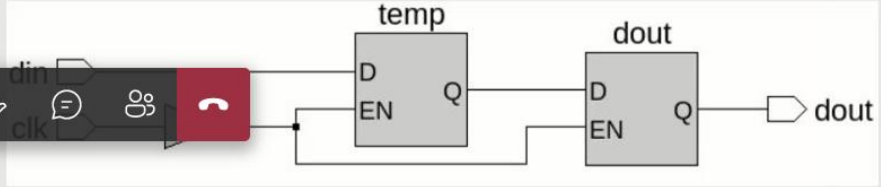



```
process ( clk )
begin
        if ( clk = '1' and clk'event ) then
                dout <= temp ;
                temp := din ;
        end if;
end process ;
```

Read Before Write




```
process ( clk , din , temp )
begin
        if ( clk = '0' ) then
                temp <= din ;
                dout <= temp ;
        end if ;
end process ;
```

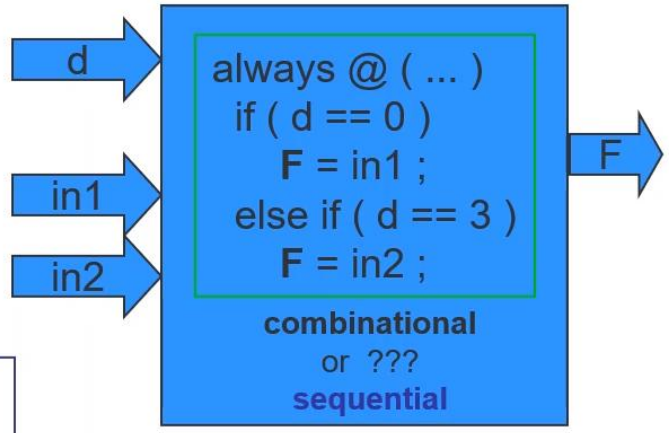


01:36:03

Good/Bad Practices in Digital Design
Un-Intentional Latches

Circuit *specifications* say that the input **"d"** only takes one of 2 values **"0" or "3"**

So the *designer thinks* that **"F"** is generated from a pure *combinational* circuit

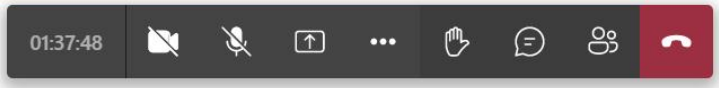


But since "d" is a 2-bit signal
→ **"d"** *can hold **4 values***

d	F
0	in1
1	??
2	??
3	in2

And nothing says **what "F" should take** *when the value of **"d" is not 0 or 3***

Therefore, we should ***keep*** *the value of "F"* ***unchanged*** when value of "d" is not 0 or 3



Good/Bad Practices in Digital Design
Un-Intentional Latches

```
process(in1)
begin
 if (in1 = con1) then
      op1 <= 3;
 elsif (in1 > con1 ) then op1 <= 5 ;
 elsif (in1 < con1 ) then op1 <= 7;
 end if;
end process;
```



Notes:

- *No else* branch in the if statement
- *all conditions* for "in1" are checked for though

Combinational

Sequential



Good/Bad Practices in Digital Design
Un-Intentional Latches

```
always @ ( select , Din )
  if ( select == 1 )
    F <= Din ;
  else
    F <= F ;
```

Notes:

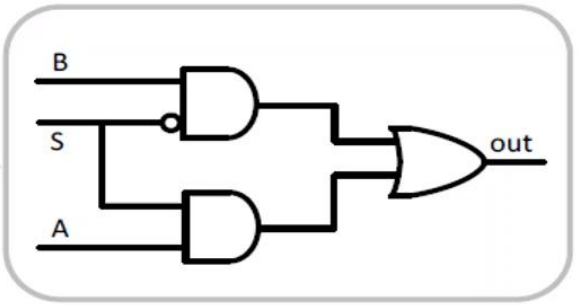
- there is an *else branch*
- ***all conditions*** of 'select' are checked for

01:39:20

El-Kholy, Mohamed (DI S...

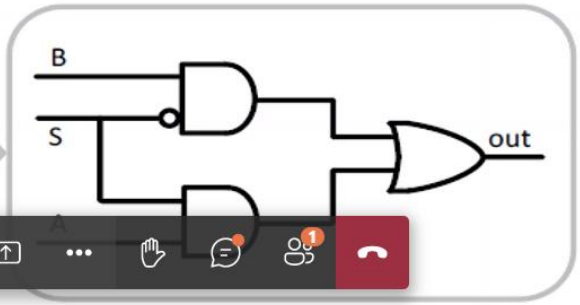
Good/Bad Practices in Digital Design
Un-Intentional Latches

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
Begin
   out = B;
   if (S==1)
       out = A;
end
endmodule
```



No Latch

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
begin
   if (S==1)
       out = A;
   else
       out =B;
end
endmodule
```



No Latch

01:40:52

Good/Bad Practices in Digital Design
Blocking vs. Non-Blocking Assignments

- **Blocking**: denoted by "**=**" token
  - Evaluation within the always block is "blocked" until this assignment is completed
- **Non-blocking**: denoted by "**<=**" token
  - Nothing is hold or blocked (parallel evaluation)

❖ Example: assume S=2 then

Blocking

```
always @ (*)
  begin
    S = 4;
    a = S;
  end
```

S=4 & a=4
(sequential)

- ❑ Evaluated and assigned in a single step
- ❑ Sequential nature
- ❑ Assignment ordering **IS** important
- ❑ S=4 "blocks" a=S to be evaluated
  - ➢ a=S has to wait for S=4 to be evaluated first

Non-Blocking

```
always @ (*)
  begin
    S <= 4;
    a <= S;
  end
```

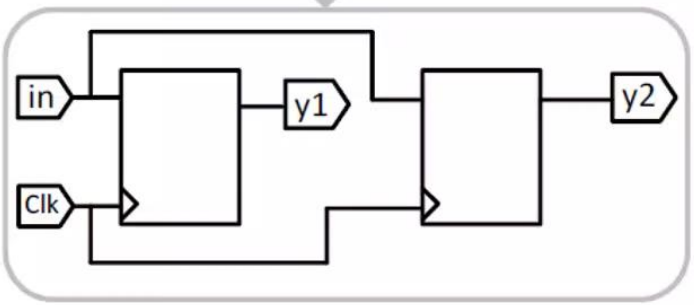
S=4 & a=2
(Parallel)

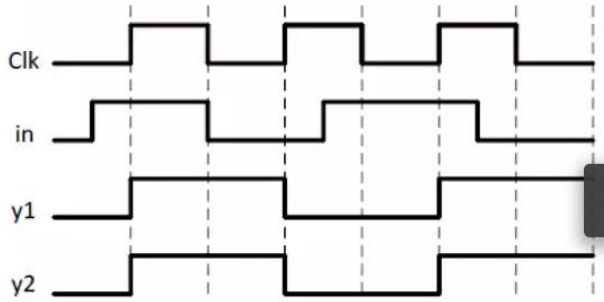
- ❑ Evaluated and assigned in two steps
  1. All RHSs are evaluated in parallel
  2. Assignments to LHSs are performed together
- ❑ They all are evaluated all at once
- ❑ Assignment ordering is **NOT** important
- ❑ S<=4 and a<=S evaluated in parallel

01:41:27

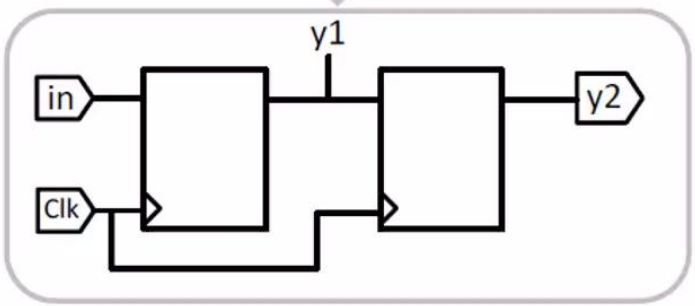
Good/Bad Practices in Digital Design
Blocking vs. Non-Blocking Assignments

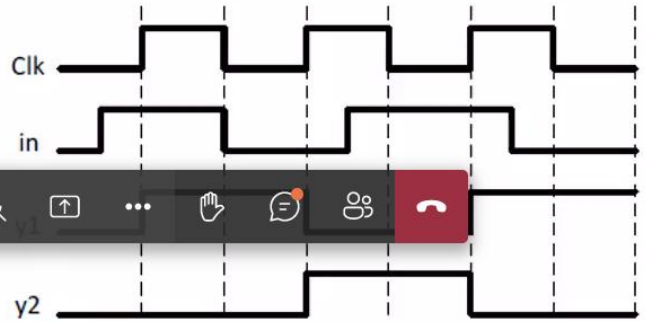
```
always @ (posedge Clk)
    begin
        y1 = in;
        y2 = y1 ;
    end
```



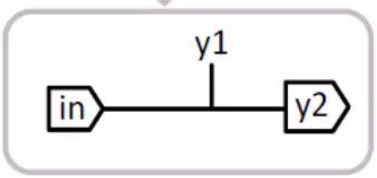



```
always @ (posedge Clk)
    begin
        y1 <= in;
        y2 <= y1 ;
    end
```






```
always @(*)
    begin
        y1 = in;
        y2 = y1 ;
    end
```



01:43:02

Good/Bad Practices in Digital Design
Blocking vs. Non-Blocking Assignments

- Use only blocking assignments for combinational logic. Why?
- Example: Accumulator: (Assume Count == 0)

```
always @ (*)
  begin
    for (k=0; k<4; k=k+1)
        Count = Count + k;
  end
```

```
Count = Count + 0;
Count = 0 + 1;
Count = 0 + 1 + 2;
Count = 0 + 1 + 2 + 3;
Result: Count = 6
```

Correct

```
always @ (*)
  begin
    for (k=0; k<4; k=k+1)
        Count <= Count + k;
  end
```

```
Count <= Count + 0;
Count <= Count + 1;
Count <= Count + 2;
Count <= Count + 3;
```

Incorrect

In multiple concurrent non-blocking assignments to a variable, the last one executes

01:45:12

Good/Bad Practices in Digital Design
Blocking vs. Non-Blocking Assignments

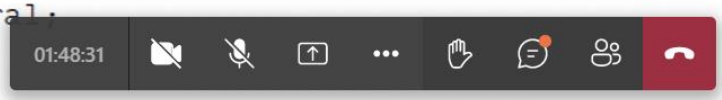
- Guideline #1: When modeling sequential logic, use nonblocking assignments.
- Guideline #2: When modeling latches, use nonblocking assignments.
- Guideline #3: When modeling combinational logic with an always block, use blocking assignments.
- Guideline #4: When modeling both sequential and combinational logic within the same always block, use nonblocking assignments.
- Guideline #5: Do not mix blocking and nonblocking assignments in the same always block.
- Guideline #6: Do not make assignments to the same variable from more than one always block.



Good/Bad Practices in Digital Design
Using Clock in Wait Statement

- **Problem**
  - If the clock event is described using *wait* statements, asynchronous resets cannot be used in a flip-flop description.
- **Solution**
  - Remove the *wait* statement, and define the clock event using *if* construct.
- **Examples**

```
 1. LIBRARY ieee;
 2. USE IEEE.STD_LOGIC_1164.ALL;
 3. USE IEEE.STD_LOGIC_unsigned.ALL;
 4. ENTITY dut IS
 5.   PORT ( clk : IN  STD_LOGIC;
 6.        q : INOUT  STD_LOGIC_VECTOR (3 DOWNTO 0):=(OTHERS=>'0'));
 7. END dut;
 8. ARCHITECTURE Behavioral OF dut IS
 9. BEGIN
10. PROCESS
11. BEGIN
12.  WAIT UNTIL (clk'event AND clk ='1');
13.  q<=q+1;
14. END PROCESS;
15. END Behavioral;
```



Good/Bad Practices in Digital Design
Using Multiple clocks in a sequential block

- **Problem**
  - Some synthesis tools might not synthesize *always* or *process* blocks that use multiple *clocks*.
- **Solution**
  - Split the sequential block into multiple blocks with each block using a single *clock*.
- **Examples**

```
module dut (in, clk1, clk2, clk3, out1, out2, out3);
  input in, clk1, clk2, clk3;
  output out1, out2, out3;
  reg out1, out2, out3;
  always   begin
    @ (posedge clk1)
    out1 <= in;
    @ (posedge clk2)
    out2<= ~in;
    @ (posedge clk3)
    out3 <= ~in;
  end
endmodule
```





Good/Bad Practices in Digital Design
Sensitivity list has a variable modified in its always block

- **Problem**
  - If a signal is present in sensitivity list, and is also modified within the same block, then it might result in inefficient simulation model. Because then, the *always* or *process* block gets evaluated continuously during simulation.
- **Solution**
  - Remove the signal from sensitivity list if it's not needed.
- **Examples**

```
1. module dut(input d, en, output reg q);
2. always@(d, en, q) begin
3.   q = d & en;
4. end
5. endmodule
```





Good/Bad Practices in Digital Design
Simulation/Synthesis Mismatch – Incomplete sensitivity list

- **Problem**
  - If a signal is not present in the sensitivity list but is read within the same block, then it might cause mismatch between the pre- and post- synthesis simulation results.
- **Solution**
  - Add the missing *read* signals in the sensitivity list of the combinational *always* construct.
- **Examples**

```
1 module dut(input [1:0] d, en, output reg q);
2 always@(d, q) begin
3   q = d[1] & en;
4 end
5 endmodule
```

  - In Simulation , q not changed when en is changed
  - In Synthesis , q is changed due to inferring combo logic





Good/Bad Practices in Digital Design
Simulation/Synthesis Mismatch – Complete sensitivity list with mis-ordered assignments

```
module code2a (o, a, b, c, d);
  output o;
  input  a, b, c, d;
  reg    o, temp;

  always @(a or b or c or d) begin
    o    = a & b | temp;
    temp = c & d;
  end
endmodule
```

temp : read before write

```
module code2b (o, a, b, c, d);
  output o;
  input  a, b, c, d;
  reg    o, temp;

  always @(a or b or c or d) begin
    temp = c & d;
    o    = a & b | temp;
  end
endmodule
```

temp : write before read

Initial Values : temp = 1 , a = 1 , b = 0 , c = 0 , d = 0

Synthesis

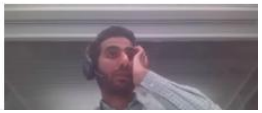
Simulation Results
O = 1
temp = 0

Simulation Results
temp = 0
O = 0

Good/Bad Practices in Digital Design
Assignment in multiple always clocks

- It's not allowed to assign same signal in different always block with different clocks
- Examples
  - always @ (posedge clk1) q <= d1 ;
  - always @ (posedge clk2) q <= d2 ;





Good/Bad Practices in Digital Design
Different types of always

always	always_comb	always_ff	always_latch
Verilog		System Verilog	
Modeling Both combinational and sequential logic	modeling synthesizable combinational logic	modeling synthesizable flip-flop logic	modeling synthesizable latch logic
it is recommended to define the sensitivity list using implicit expression list always @(*) instead of explicitly defining signals. This ensures that the sensitivity list is always complete and simulation behavior is correct.	always_comb is preferred over always @(*).	restricts the procedure to contain only one event control without any blocking timing controls.	Sensitivity list is automatically built.
		• Variables written on the left-hand side of assignments cannot be written by other process • Most of the software tools perform additional checks to ensure code within the procedure models correct behavior	
			is triggered once at time zero so that output of the block aligns with the inputs.

01:54:05

Good/Bad Practices in Digital Design
Different types of always

```
module dff1 (
  output bit_t q,
  input  bit_t d, clk, rst_n);

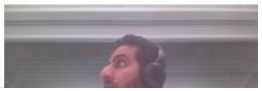
  always_ff @(posedge clk, negedge rst_n)
    if (!rst_n) q <= 0;
    else        q <= d;
endmodule
```

```
module latch(input  logic       clk,
             input  logic [3:0] d,
             output logic [3:0] q);

  always_latch
    if (clk) q <= d;

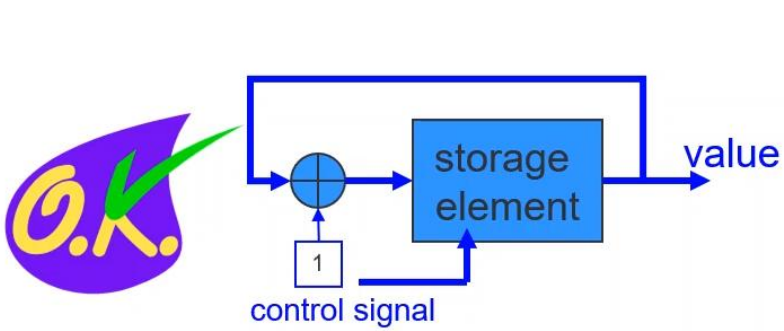
endmodule
```

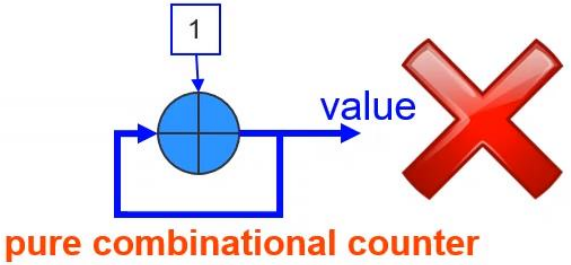
01:58:10



Combinational/Sequential Loops

- Pure Combo loop is leading to oscillating output , storage elements should be used





- Formation of a loop from the flip-flop output, through a combinational logic, to its asynchronous inputs leads to unexpected design behavior.

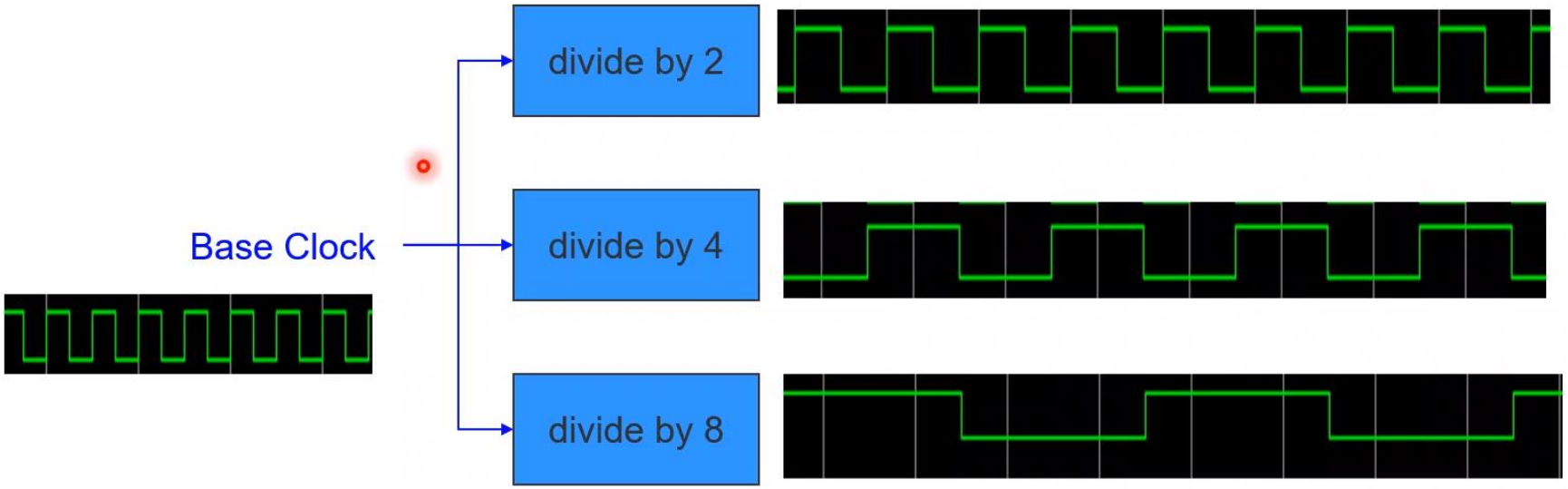
```
1. module dut (clk, load, d, q);
2.  input  clk, load, d;
3.  output q;
4.   reg    tmp;
5.  always @(posedge clk or negedge load) begin
6.    if (!load) tmp<= 0;
7.   else tmp <= d;
8.  end
9.  assign q = tmp;
10. endmodule
```

01:58:47

El-Kholy, Mohamed (DI S...

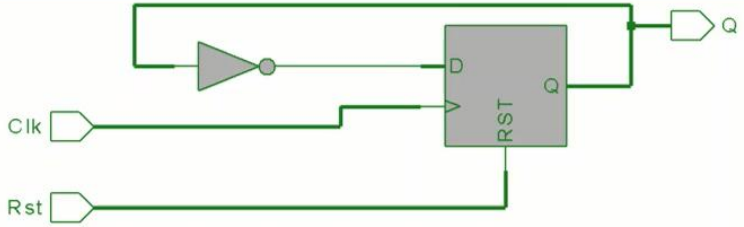
Clock Dividers

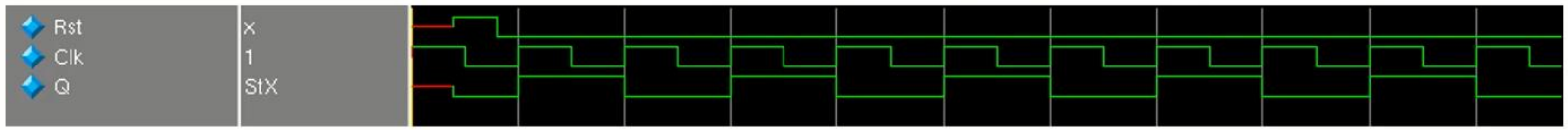
- Clock Dividers (*i.e. frequency dividers*)



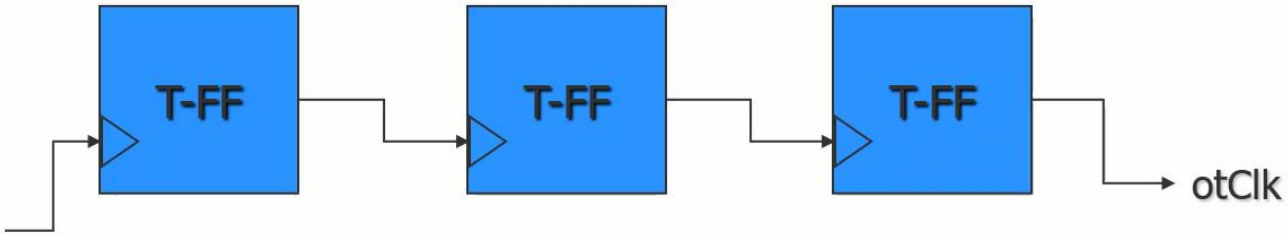
Clock Dividers

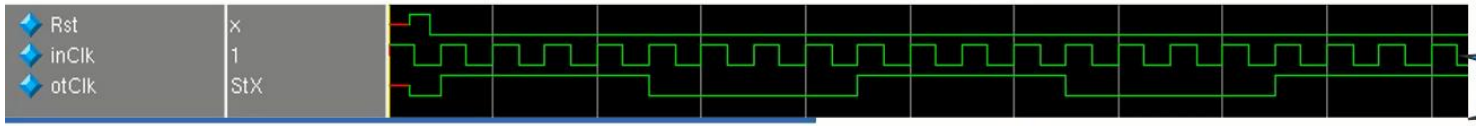
- T-FF (toggle)





- One implementation is to use *Toggle flip flops*

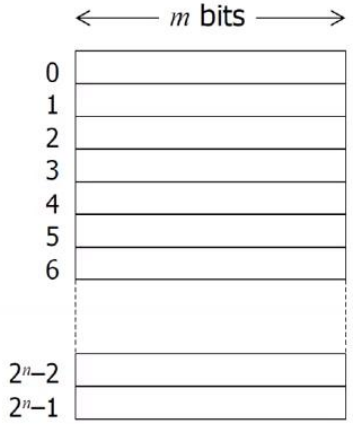




otClk is 8 times *slower* than inClk

Memory

- A memory is an array of storage locations
  - Each with a unique address
  - n address bits ⇒ 2^n locations
  - All locations the same size
  - 2^n × m bit memory
- Memory Types
  - RAM
  - ROM
  - FIFO

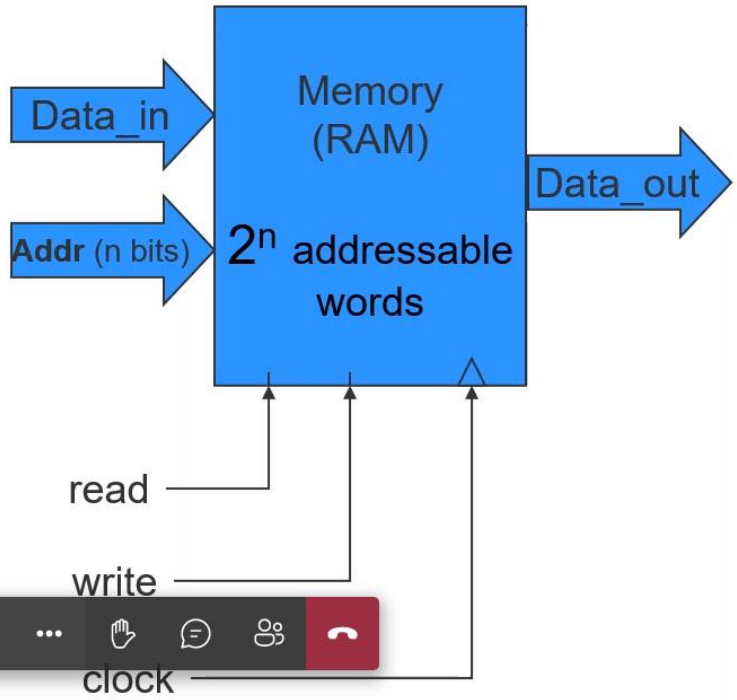


RAM	ROM	FIFO
Random Access Memory Can Write/Read in/from any address/location	Read-only Memory - Data in ROM can only be read	FIFO - first in first out Address is internal pointer which increase each write operation and decrease each read operation

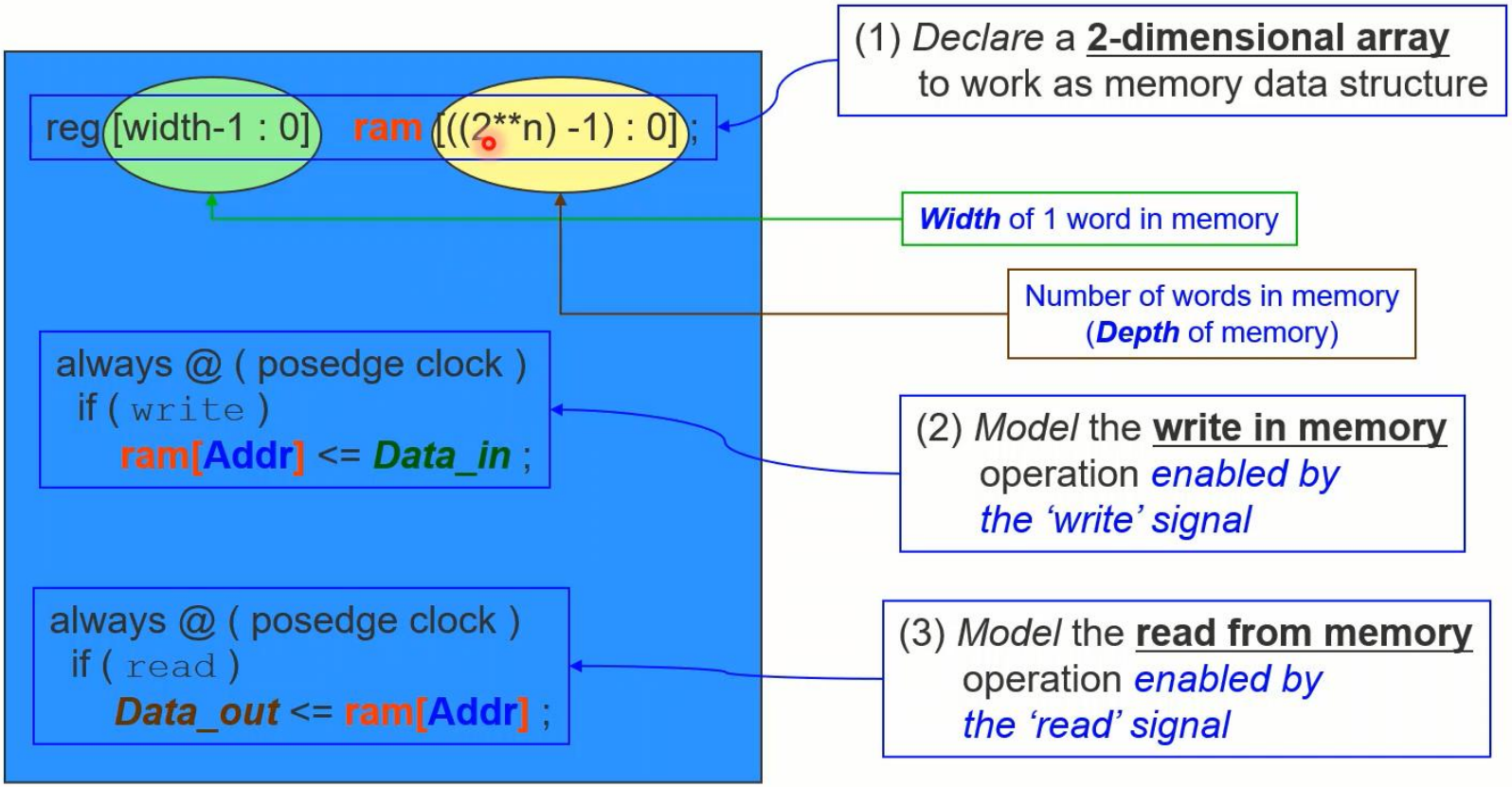


RAM

Data_in	**Vector of bits to be written into the memory**
Data_out	**Vector of bits to be read out of the memory**
Addr	**Vector of bits used as index to access memory words**
clock	**Clock triggering the write and read operations**
read	**signal to enable the read operation**
write	**signal to enable the write operation**

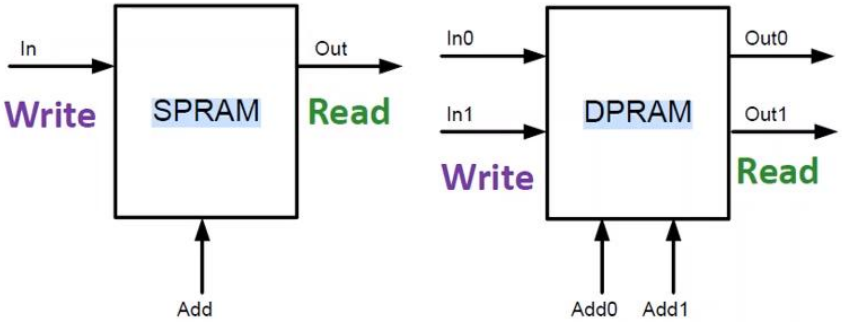


RAM


reg [width-1 : 0] ram [((2**n) -1) : 0];
always @ (posedge clock)
if (write)
ram[Addr] <= Data_in ;
always @ (posedge clock)
if (read)
Data_out <= ram[Addr] ;
(1) Declare a 2-dimensional array to work as memory data structure
Width of 1 word in memory
Number of words in memory (Depth of memory)
(2) Model the write in memory operation enabled by the 'write' signal
(3) Model the read from memory operation enabled by the 'read' signal

RAM

- RAM Variations
  - Signle Port (SPRAM) / Dual Port (DPRAM)
    - single port memory has single set of address and controls, and it can only have single access (read/write) at a time.
      dual port memory has two sets of address and controls, and it can have two simultaneous access to two different memory locations.




```
// CONTINUED
// Memory Write : when we = 1, cs = 1
always @ (posedge clk)
  begin  : MEM_WRITE_READ
   if ( cs_0 && we_0 )
      mem[address_0] <= data_0;
  else if (cs_0 && !we_0 && oe_0)
      data_out_0 <= mem[address_0];
 end
  if ( cs_1 && we_1 )
      mem[address_1] <= data_1;
   else if (cs_1 && !we_1 && oe_1)
      data_out_1 <= mem[address_1];
  end
endmodule
```

  - Single / Dual Clock
  - Synchronous / Asynchronous Output
  - SRAM/DRAM
    - Static RAM (SRAM) : Stores data so long as power is supplied
    - Dynamic RAM (DRAM) : Needs to be periodically refreshed



ROM

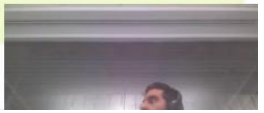
- Asynchronous(Combinational) / Synchronous Output

```
module ROM_sevenSegment
#(
    parameter addr_width = 16, // store 16 elements
              addr_bits = 4, // required bits to store 16 elements
              data_width = 7 // each element has 7-bits
)
(
    input wire clk,
    input wire [addr_bits-1:0] addr,
    output reg [data_width-1:0] data  // reg (not wire)
);


always @*
begin
    case(addr)
        4'b0000 : data = 7'b1000000; // 0
        4'b0001 : data = 7'b1111001; // 1
        4'b0010 : data = 7'b0100100; // 2
        4'b0011 : data = 7'b0110000; // 3
        4'b0100 : data = 7'b0011001; // 4
        4'b0101 : data = 7'b0010010; // 5
        4'b0110 : data = 7'b0000010; // 6
        4'b0111 : data = 7'b1111000; // 7
        4'b1000 : data = 7'b0000000; // 8
        4'b1001 : data = 7'b0010000; // 9
        4'b1010 : data = 7'b0001000; // a
        4'b1011 : data = 7'b0000011; // b
        4'b1100 : data = 7'b1000110; // c
        4'b1101 : data = 7'b0100001; // d
        4'b1110 : data = 7'b0000110; // e
        default : data = 7'b0001110; // f
    endcase
end

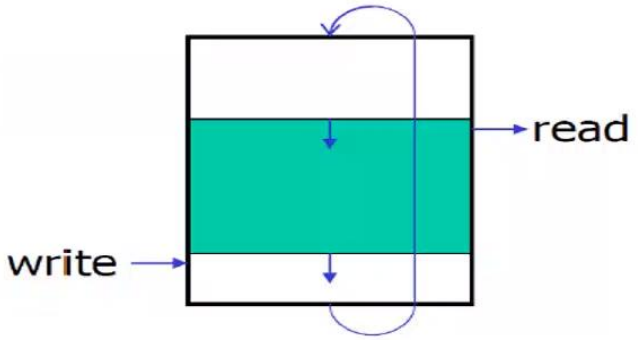
endmodule
```

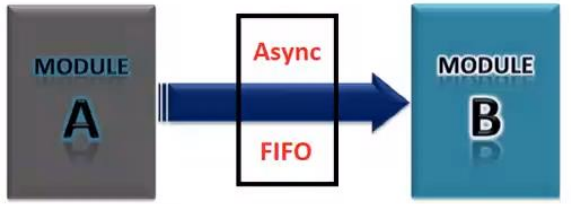
02:09:15

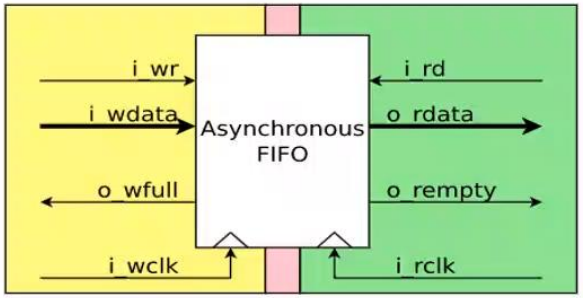


FIFO

- First-In/First-Out buffer
- Circular buffer
- FIFO Usages
  - Synchronization between fast transmitter and slow receiver
  - to buffer the data which is not read by the slower module.
- Interface
  - Write , Read Clocks
  - Write , Read Enables
  - Write , Read Data
  - Two Important Flags
    - Full : to avoid the case of invalid request of write operation when the FIFO is already full.
    - Empty : to avoid the case of invalid request of read operation when the FIFO is already empty.
- Addresses
  - Internal write , read pointers







Press Ctrl+Shift+M to unmute your microphone.
02:10:07

FIFO Design

- Depth Calculation

Example 1 : f_A > f_B with no idle cycles in both write and read.

— Writing frequency (f_A)= 80MHz.
— Reading Frequency (f_B) = 50MHz.
— Burst Length = No. of data items to be transferred = 120.
— There are no idle cycles in both reading and writing which means that, all the items in the burst will be written and read in consecutive clock cycles.

Solution :-

1) Time required to **write** one data item = $\frac{1}{80\text{MHz}}$ **= 12.5 nsec**
2) Time required to **read** one data item = $\frac{1}{50\text{MHz}}$ **= 20 nsec**
3) Time required to **write** all the data in the burst = **120 * 12.5 = 1500 nsec**
4) Number of data items can be read in a duration of 1500 **nsec** = $\frac{\mathbf{1500\ nsec}}{20\ \text{nsec}}$ = 75
5) The remaining no. of bytes to be stored in the FIFO = 120 – 75 = 45.

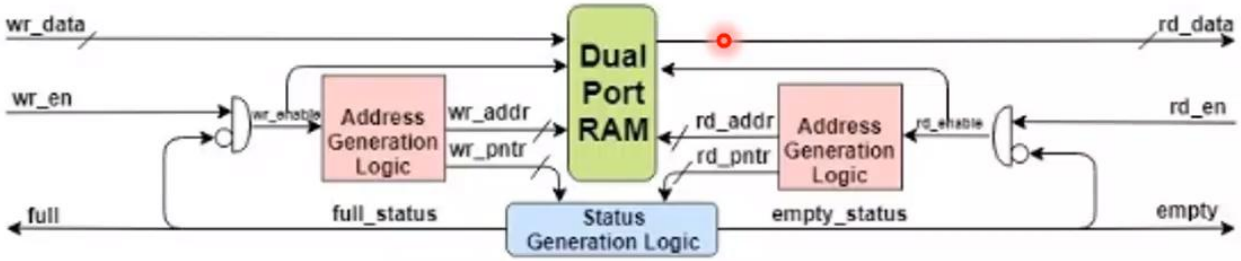
The minimum depth of the FIFO should be 45.

Press Ctrl+Shift+M to unmute your microphone.

Depth Calculation
Example 1 : > with no idle cycles in both write and read.
— Writing frequency ( )= 80MHz.
— Reading Frequency ( ) = 50MHz.
— Burst Length = No. of data items to be transferred = 120.
— There are no idle cycles in both reading and writing which means that, all the items in the burst will be written and read in consecutive clock cycles.
Solution :-
1) Time required to write one data item = 1/80MHz = 12.5 nsec
2) Time required to read one data item = 1/50MHz = 20 nsec
3) Time required to write all the data in the burst = 120 * 12.5 = 1500 nsec
4) Number of data items can be read in a duration of 1500 nsec = 1500 nsec / 20 nsec = 75
5) The remaining no. of bytes to be stored in the FIFO = 120 – 75 = 45.

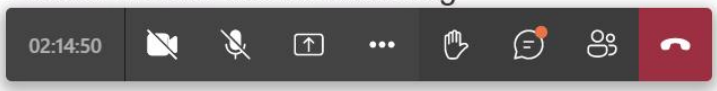
FIFO Design

- Architecture



Synchronous FIFO architecture

- Challenge
  - Problem
    - the common problem is controlling of FULL and EMPTY status signals. In Synchronous FIFO where the read and write rates are same, we can simply count the total number of variables in FIFO. Increase the count whenever the write operation is performed, decrease the count whenever the read operation is performed and leave the count unchanged when read and write operations are performed simultaneously.
    - But it's not the case with Asynchronous FIFO. As here the rate of data write and read are different so we can't just keep increasing and decreasing the count for data write and read respectively.
  - Solution to the problem:
    - On doing successive operation of read and write we can met two conditions of intersection of locations stored in read and write pointer. First when the rate or reading is higher than that of rate of write and read pointer catches up write pointer, this is the condition of EMPTY FIFO. Second is when rate of write is higher than that of read and write pointer catches up read pointer by already completing a round of FIFO.
    - As in both of these cases the read and write pointer are containing same locations, so we need to distinguish these two cases. To distinguish these two cases we can use an extra flag

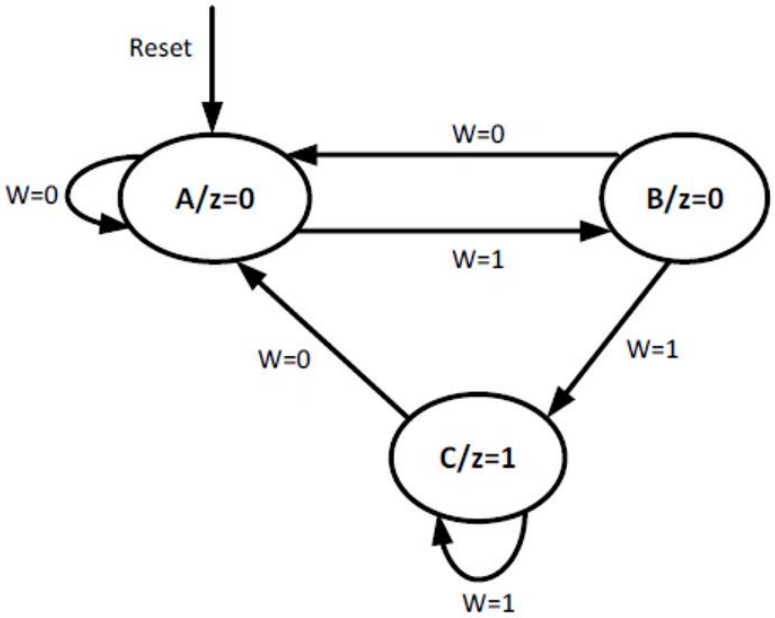


FIFO Design

- Homework
  - Deliver FIFO module
    - module FIFO #(parameter ADDR_WIDTH=5, DATA_WIDTH=8) (data_out, data_in, clk_read, clk_write, enqueue, dequeue);
    - Will check coding guidelines issues
  - Deliver Test Bench
    - Module FIFO_TB ();
    - Try to Cover all scenarios
    - Example sequence
      - Trying to read while FIFO is empty
      - Write 2 words and read them afterwards
      - Trying to read while FIFO is empty
      - Write till making it full again (circular writing)
      - Reading till making it empty gain (circular reading)
  - Submit your project in OneDrive Folder
    - Create SubFolder : proj_<ID>
    - Will send excel sheet having the ID number of each student

Finite State Machine (FSM)

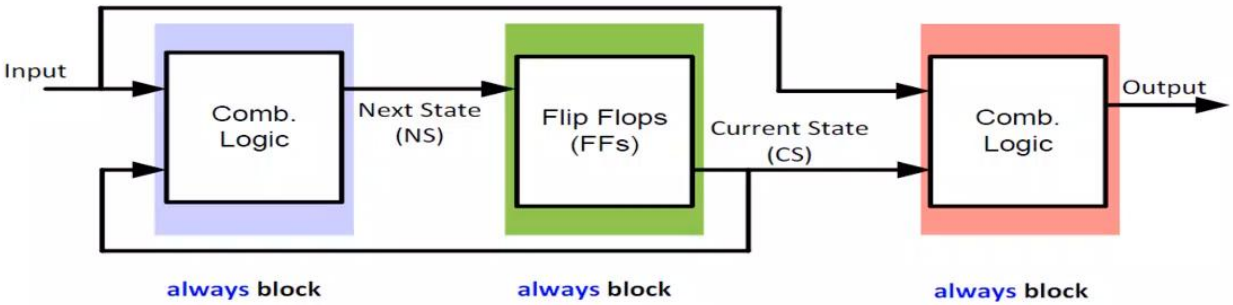
- Used to implement control sequencing
- An FSM is defined by
  - set of inputs : W
  - set of outputs : z
  - set of states : A , B , C
  - initial state : A
  - transition function
  - output function
- States are steps in a sequence of transitions
- There are “Finite” number of states.

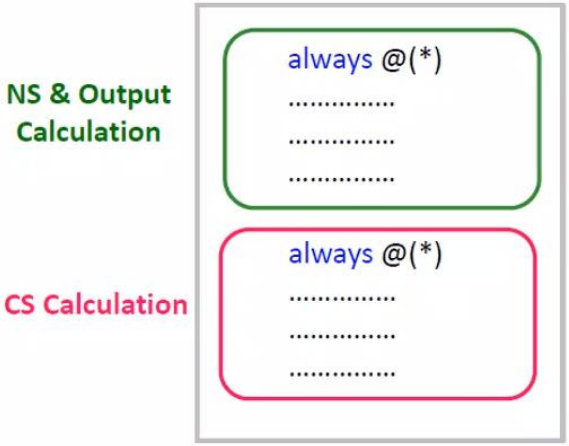


02:20:03

Finite State Machine (FSM)

- Two types:
  - **Mealy:** Output depends on the "current state" and the "input"





Output: reg

```
module mealy (Clock, w, Resetn, z);
 input Clock, w, Resetn ;
 output reg z ;
 reg CS, NS;
 parameter A = 1'b0, B = 1'b1;
 always @(w, CS)
  case (CS)
    A:  if (w == 0)
          begin
            NS = A;   z = 0;
          end
        else
          begin
            NS = B;   z = 0;
          end
   B:  if (w == 0)
          begin
            NS = A;   z = 0;
          end
       else
          begin
            NS = B;   z = 1;
          end
  endcase
 always @(posedge Clock, negedge Resetn)
  if (Resetn == 0)
   CS <= A;
  else
   CS <= NS;
endmodule
```

NS & Output Calculation

Combinational (Blocking)

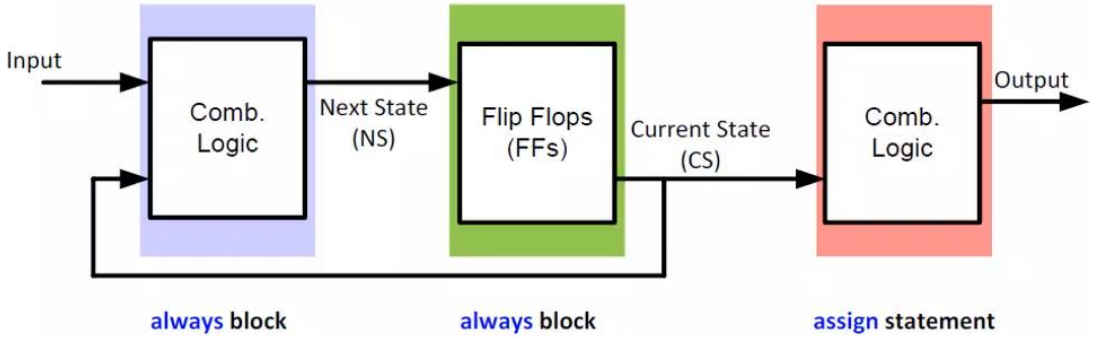
CS Calculation

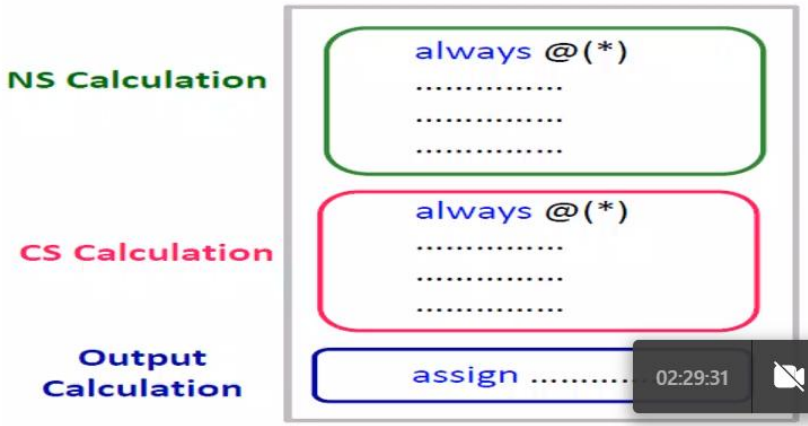
Sequential

02:26:01

Finite State Machine (FSM)

- Two types:
  - **Moore:** Output depends only on the "current state"






```
module moore (Clk, w, Resetn, z);
 input Clk, w, Resetn;
 output z;
 reg [1:0] CS, NS;
 parameter A = 2'b00, B = 2'b01, C = 2'b10;

 always @(w, CS)
 begin
  case (CS)
    A: if (w == 0)  NS = A;
       else  NS = B;
    B: if (w == 0)  NS = A;
       else  NS = C;
    C: if (w == 0)  NS = A;
       else  NS = C;
   default:   NS = 2'bxx;
  endcase
 end
 always @(posedge Clk, negedge Resetn)
 begin
  if (Resetn == 0)
    CS <= A;
  else
    CS <= NS;
 end

 assign z = (CS == C);
endmodule
```

Output: wire

NS Calculation

CS Calculation

Output Calculation

Combinational (Blocking)

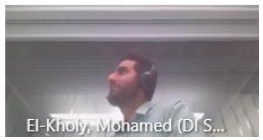
Sequential (Non-Blocking)

Finite State Machine (FSM)

- Dead End State
  - **Problem**
    - FSM states without any outgoing transitions reaches a dead-end;
  - **Solution**
    - Review the code to avoid any dead-end FSM state.
  - **Examples**
    - In the following example, state *ST4* reaches a dead-end as no outgoing transition is defined for ST4.

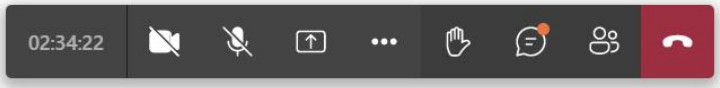
```
module dut (clk, rst, a, b, p);
input clk;
input rst;
input a, b;
output p;
reg p;
parameter ST0=1, ST1=2, ST2=4, ST3=8, ST4=16; reg [4:0] state;
always @(posedge clk or posedge rst)
begin
  if (rst)
   state = ST0;
  else begin
   case (state)
   ST0 : begin state = ST1; p = a; end
   ST1 : begin state = ST2; p = a; end
   ST2 : begin state = a ? ST0 : ST4; p = a; end
   ST3 : begin state = ST1; p = b; end
   endcase
  end
```





Tips

- Understand synthesis tool capabilities and limitations
  - translate_off and translate_on directives instruct synthesis tools to ignore portions of a Verilog code





Digital Design Metrics

- **Speed**
  - Throughput
    - The amount of data that is processed per clock cycle (bits per second)
  - Latency
    - The time between data input and processed data output (clock cycle)
  - Timing
    - The logic delays between sequential elements (clock period)
    - When a design does not meet the timing it means the delay of the critical path is greater than the target clock period
- **Area**
- **Power**
  - Dynamic
    - Switching activity

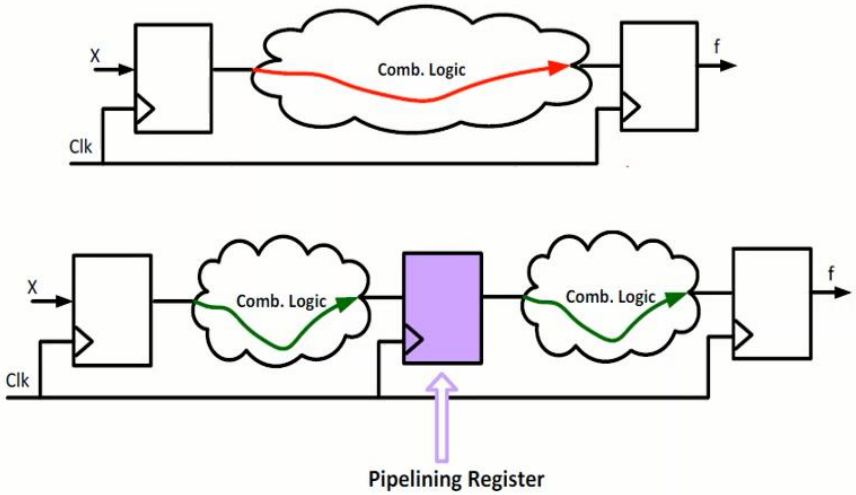
$$P_{Dynamic} = alpha * f * C_L * Vdd^2$$

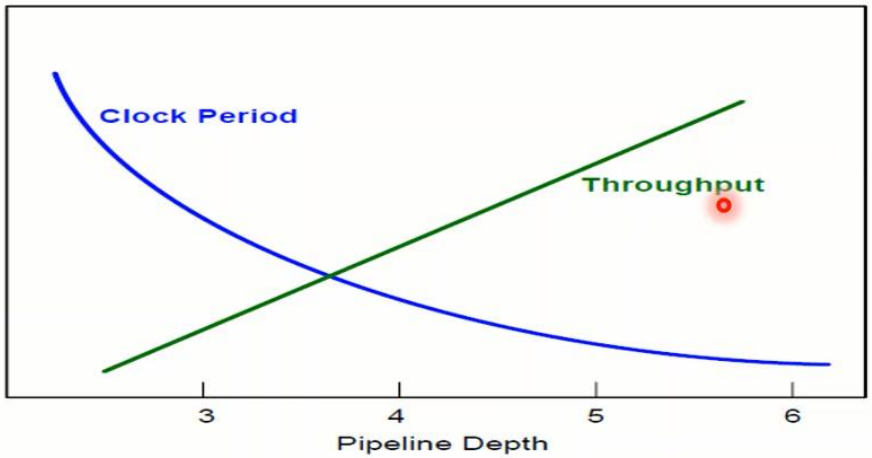
  - Static (leakage)



Design Optimization Techniques

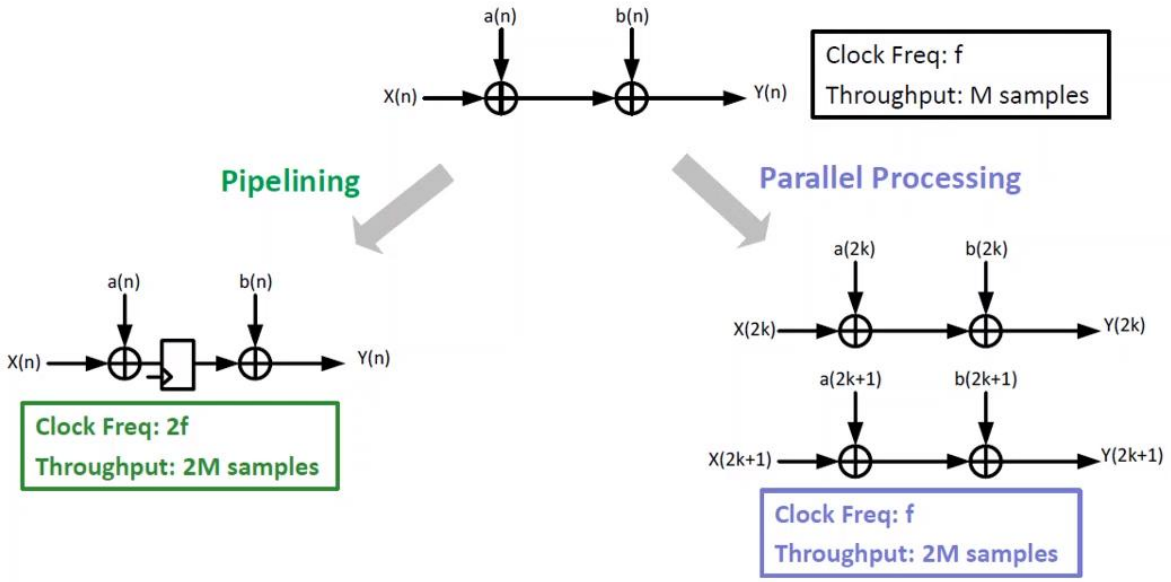
- Pipelining
  - Comes from the idea of a water pipe: continue sending water without waiting the water in the pipe to be out
  - Advantageous:
    - Reduction in the critical path
    - Higher throughput (number of computed results in a give time)
    - Increases the clock speed (or sampling speed)





Design Optimization Techniques

- Parallelism
  - the same hardware is duplicated to
    - Increases the throughput without changing the critical path
    - Increases the silicon area



02:44:51

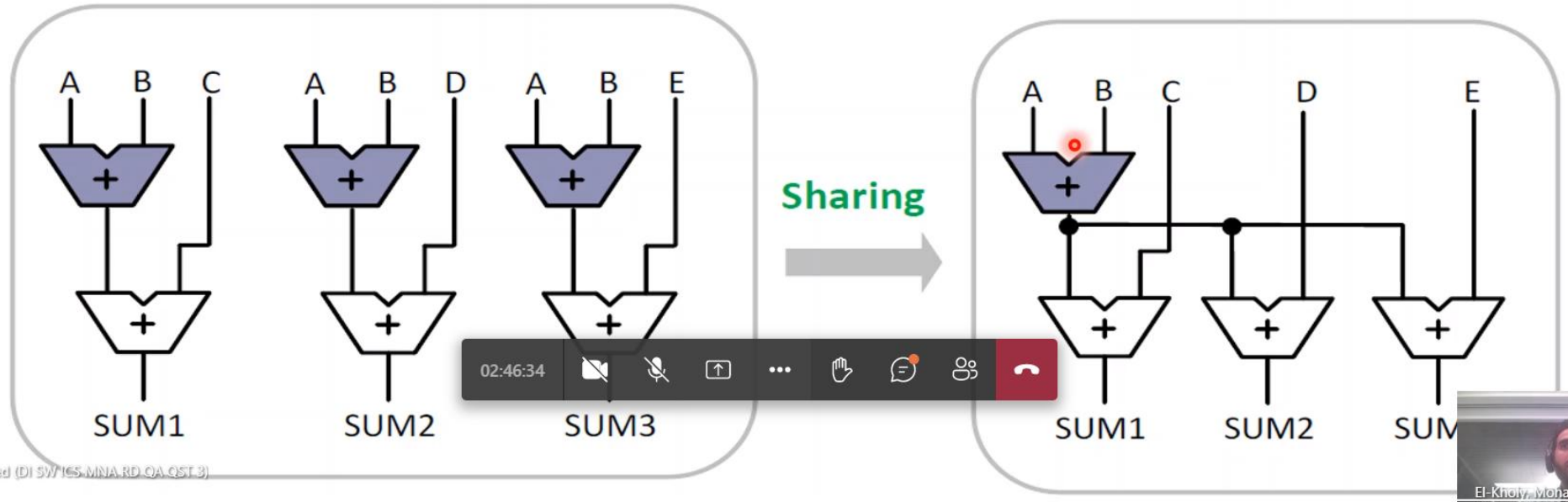
El-Kholy, Mohamed (DI S...

Design Optimization Techniques

- Resource Sharing (**Area Reduction Technique**)
  - Resource Sharing is used where there are functional blocks that can be used in other areas of the design or even in different modules

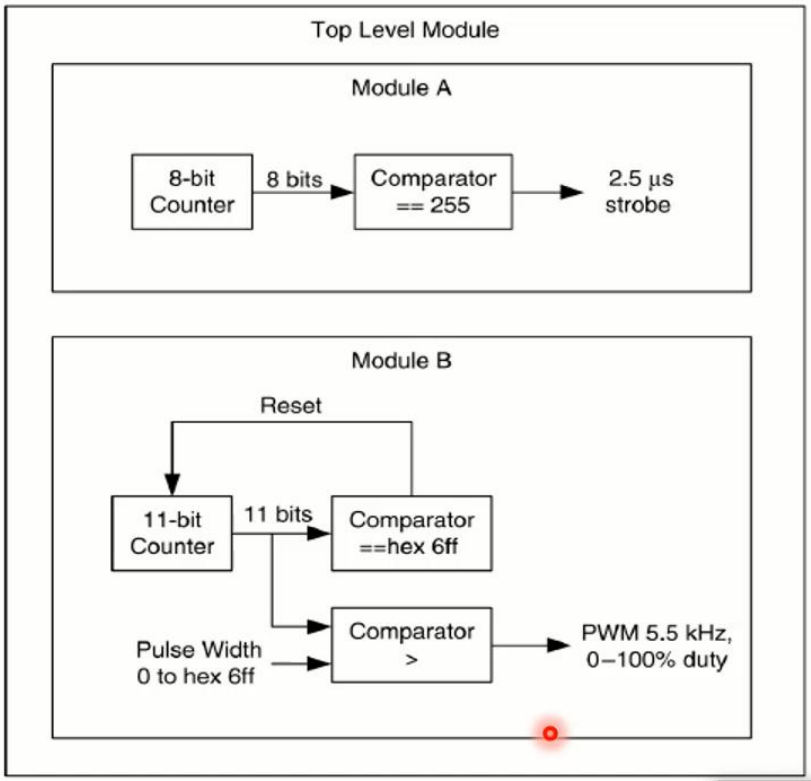
```
SUM1 <= A+B+C;
SUM2 <= A+B+D;
SUM3 <= A+B+E;
```

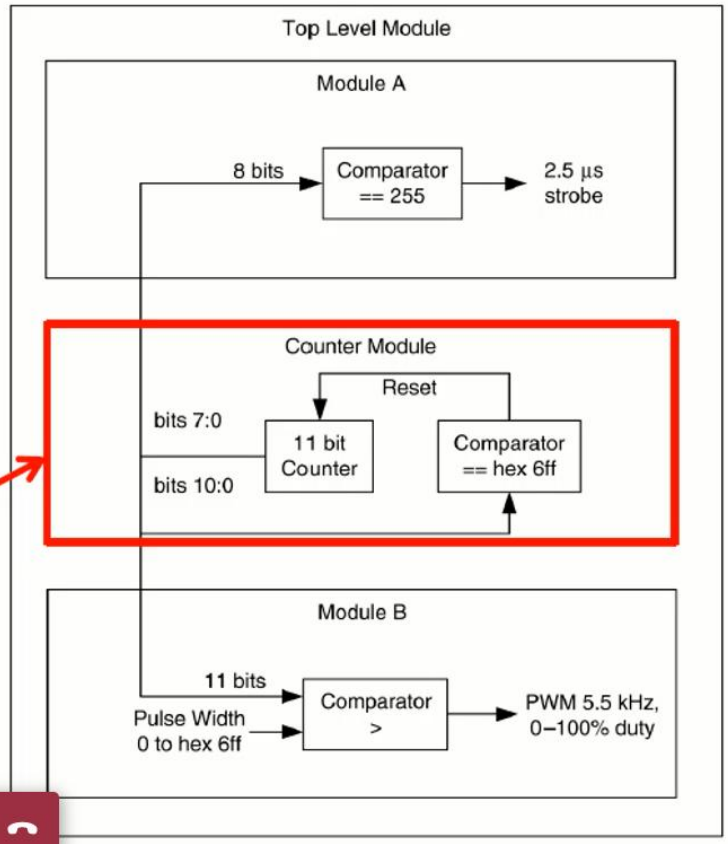
```
assign  tmp = A+B;
SUM1 <= tmp +C;
SUM2 <= tmp +D;
SUM3 <= tmp +E;
```



Design Optimization Techniques

- Resource Sharing (**Area Reduction Technique**)



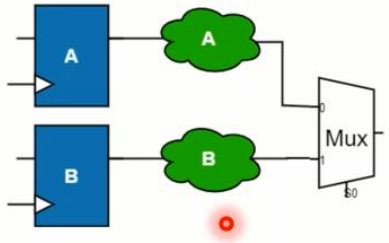


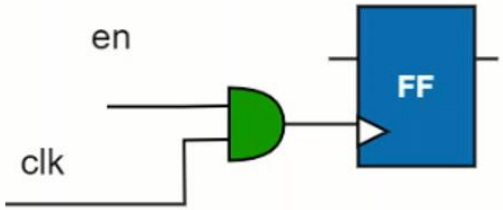
Shared Part

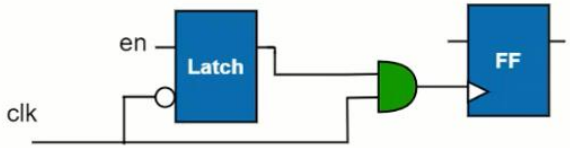
El-Kholy, Mohamed (DI SW ICS MNA RD QA QST 3)

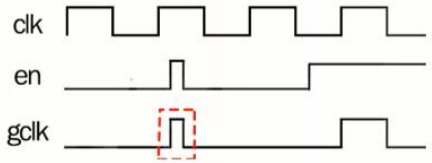
Design Optimization Techniques

- Clock Gating
  - the most famous and most efficient technique to reduce dynamic power.
  - There are situations when we don't need the FF to capture the data
    - Examples
      - If the mux selection is 0 The MUX won't select FF B output. So we don't need FF B to sample the data thus dissipating power in itself and in the combinational circuit B between it and the MUX.
      - When the input is not changing: There is some power that gets wasted within the FF even if its input is not changing so it's important to stop sending the clock to the FF to stop this power dissipation.
  - Implementation
    - And gate
      - Using just an AND gate has several issues. If a glitch happens in the enable signal during the high period of the clock, this will lead to an unintended clock edge
    - Latched and gate









Design Optimization Techniques

- Disadvantages of clock gating
  - Clock gating adds an additional area to the design
  - The CG itself consumes power.
    - A good designer should use gating when necessary. If the gated blocks are enabled “active” most of the time, then the gating is unnecessary and the design will now be taking more area and power (due to the ICG) without use!

Resources

- Verilog and SystemVerilog Gotchas - 101 Common Coding Errors and How to Avoid Them
  - Stuart Sutherland , Don Mills
- RTL Coding Styles That Yield Simulation and Synthesis Mismatches
  - Don Mills , Clifford E. Cummings
- Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!
  - Clifford E. Cummings
- 100 Power Tips for FPGA Designers
  - Evgeni Stavinov
- Xilinx Recommended Coding Style
- Altera/INTEL Recommended Coding Style
- Verilog Coding Style for Efficient Digital Design
  - Kapil Batra , Mohammad Suhaib Husain
- ASIC & FPGA Chip Design
  - Mahdi Shabany

Thank You

02:54:34

El-Kholy, Mohamed (DI SW ICS MNA RD QA QST 3)

